STABILITY AND CONVERGENCE: A LAB

Why a time step can be too large and a grid too coarse; Euler's method and the heat equation as the warm-up; the rubber band blown up two ways, a stiffness–time-step map, and refinement in space and time measured on IB2d



Laura A. Miller

Department of Mathematics, University of Arizona

Using IB Software • Part II • Chapter 4d

Claude (Anthropic) was used to organize, combine, and format more than twelve years (2013–2026) of the instructor's tutorial notes from courses and workshops at several universities.

WHAT YOU WILL BE ABLE TO DO AFTER THIS CHAPTER

- Say what convergence, order and stability mean for a numerical method, and tell an unstable run from an inaccurate one.
- Reproduce the course's two warm-ups: Euler's method on $y' = -15y$ (one multiplier per step decides everything) and the explicit heat equation (stable only for $r = \Delta t/\Delta x^2 \leq 1/2$; convergence in space and in time; the order measured).
- Blow up IB2d's rubber band two ways on purpose (a time step of 1, a band 1000 times stiffer), read the error each port gives, and map which stiffness–time-step pairs survive.
- Refine an IB2d example properly: the points stay at $\Delta s \approx \Delta x/2$, `input2d` and the generator agree, the spring constant is rescaled so the band is the same band; then measure how fast the answer converges and what it costs.
- Run the lab in Python on a laptop (`examples/IB2d/Stability_Lab`: a generator with the grid and stiffness as arguments, a sweep script, an analysis script) and write the same protocol for an example of your own.

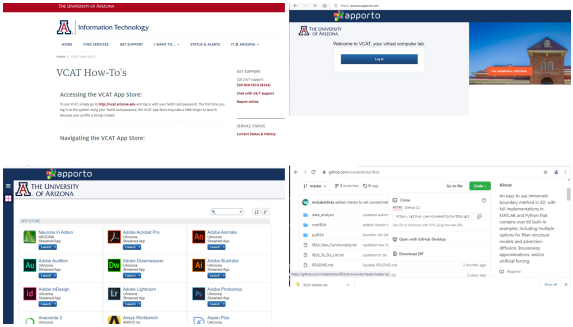
> Everything measured here ran in October 2026 with the Python IB2d on a desktop PC under WSL (73 runs, most 2–90 s; the 256^2 runs 3–8 minutes). The numbers are in the tables; the scripts that made them are in the repository, so the tables can be remade on another machine.

WHERE THIS CHAPTER COMES FROM

- "Lecture 19: Stability and convergence" (UA course, 2021, edited 2023; 40 slides): the three ideas, Euler's method by hand and in `stiff_ode_example.m`, the CFL condition, the heat equation in `heat_equation.m` (space, then time), rate and order of convergence, the IB rules of thumb, and the homework-5 lab on the rubber band (blow it up with Δt, blow it up with stiffness, refine $32 \to 64$). The only stability lab in twelve years of material.
- Lecture 12 (2021–23), its numerics block (slides 8–21): the same ideas in short form, plus "IBAMR instabilities": the time step needed is inversely proportional to the force applied, the forcing term (not CFL) is what blows up, so avoid large accelerations and very stiff springs.
- The MATLAB exercise files (`stiff_ode_example.m`, `heat_equation.m`): in the repository under `examples/support/matlab-exercises`, corrected, with Python versions that give the same numbers (chapter 5 runs them; the Octave-against-Python check is in the folder's README, Appendix A).
- The course's remote-desktop route (UA's VCAT virtual lab, MATLAB, downloading IB2d): kept on one slide; chapters 1 and 4a are the current set-up.

What is new here: every figure remade from the scripts; the measured orders; the IB2d lab rewritten for the Python port with a parametrised generator, a 41-run stiffness–time-step map, a five-grid refinement with the tension kept and a four-step time-step study; the area loss of a closed band measured against grid and points per cell (it is the method's, not the spacing's); the inverse relationship checked rather than stated.

THE COURSE'S LAB SET-UP, THEN AND NOW



Lecture 19, slides 7–10 (2021): UA's VCAT virtual computer lab (apporto) with MATLAB; IB2d downloaded from GitHub as a zip.

Then (2021): log in to VCAT with a NetID ("this might take a while the first time"), launch MATLAB, download and extract `ib2d.zip` to Documents or the Desktop, download VisIt from https://visit-dav.github.io/visit-website/releases-as-tables/#latest, make a `MATH481/HW5` folder with `heat_equation.m` and `stiff_ode_example.m`, unzip `Rubberband_HW5` into `Example_Standard_Rubberband`, and change MATLAB's folder to it (or type the path).

Now: the two exercise files and the rubber band are in this repository (`examples/support/matlab-exercises`, `examples/IB2d/Stability_Lab`); IB2d in MATLAB or Python as in chapter 4a; VisIt as in chapter 4b; Puma as in chapter 1. The VCAT route still exists (https://it.arizona.edu/documentation/vcat-how-tos) for a machine without MATLAB.

NUMERICAL ANALYSIS: THREE IDEAS

Numerical analysis is not only the design of numerical methods but also their analysis. Three central concepts:

- convergence: whether the method approximates the solution (the numerical solution approaches the exact one as the step size $h \to 0$),
- order: how well it approximates the solution (how fast the error shrinks with h),
- stability: whether errors are damped out.

> Numerical instability. Typically an algorithm would approach the right solution in the limit if there were no round-off or truncation errors; but depending on the method, errors can be magnified instead of damped, and the error grows exponentially. An unstable run is not "a bit off": it is garbage after a few steps, and usually ends with `NaN` or an error.

Two things to keep apart from the start: a stable but inaccurate run (coarse grid, large but admissible step: smooth, wrong numbers) and an unstable run (oscillations that grow, then overflow). The first is fixed by refinement; the second by a smaller step or a softer force, as the lab shows.

THE MOTIVATING EXAMPLE: $y' = -15y$

Consider the initial value problem

$$y'(t) = -15\,y(t), \qquad t \geq 0,\ y(0) = 1, \qquad \text{with exact solution} \qquad y(t) = e^{-15t} \to 0 \text{ as } t \to \infty.$$

We seek a numerical solution that exhibits the same behaviour. The figure in the lecture (reproduced on slide 9) illustrates the numerical issues for integrators applied to this equation:

1. Euler's method with step size $h = 1/4$ oscillates wildly and quickly exits the range of the graph;
2. Euler's method with half the step size, $h = 1/8$, produces a solution within the graph boundaries, but oscillates about zero;
3. the trapezoidal method (the two-stage Adams–Moulton method)

$$y_{n+1} = y_n + \tfrac{1}{2}h\left(f(t_n, y_n) + f(t_{n+1}, y_{n+1})\right)$$

   gives a much better result (slide 12).

The equation is stiff: its own time scale ($1/15$) is far shorter than the interval you care about (t up to 1), so the step size is dictated by stability, not by the accuracy you would need to draw e^{-15t}.

EULER'S METHOD BY HAND: $h = 1/4$

$y'(t) = -15\,y(t) = f(t, y(t))$, $y(0) = 1$. Let $h = \frac{1}{4}$.

$$\begin{aligned} y_1 &= y_0 + h\,f(t_0, y_0) = y_0 + h\,(-15y_0) \\ &= 1 + \tfrac{1}{4}(-15 \cdot 1) \\ &= 1 - \tfrac{15}{4} = -2.75 \end{aligned}$$

$$\begin{aligned} y_2 &= y_1 + h\,f(t_1, y_1) = y_1 + h\,(-15y_1) \\ &= -2.75 + \tfrac{1}{4}(-15 \cdot (-2.75)) \\ &= 7.5625 \end{aligned}$$

$$y_3 = -20.796875, \qquad y_4 = 57.19140625$$

The exact solution at those times: 0.0235, 0.00055, $1.3 \cdot 10^{-5}$, $3.1 \cdot 10^{-7}$.

Why. Each step multiplies y by the same number:

$$y_{n+1} = y_n - 15h\,y_n = (1 - 15h)\,y_n,$$

$$y_n = (1 - 15h)^n.$$

- $|1 - 15h| < 1 \iff 0 < h < \frac{2}{15} = 0.133$: the method decays (stable);
- $-1 < 1 - 15h < 0 \iff \frac{1}{15} < h < \frac{2}{15}$: decays, but changes sign every step;
- $h > \frac{2}{15}$: $|1 - 15h| > 1$, growth by that factor per step. At $h = 1/4$ the factor is -2.75.

For $y' = \lambda y$ in general the factor is $1 + h\lambda$; Euler is stable where $|1 + h\lambda| < 1$, the disc of radius 1 about -1 in the complex $h\lambda$ plane. A stiff system is one whose fastest λ sets h.

STIFF_ODE_EXAMPLE.M, AS HANDED OUT

```
function stiff_ode_example(dt, tfinal)
% This example numerically solves the stiff ode y'(t) = -15*y(t)
      using Euler's
% method. This has an exact solution given by y(t) = exp(-15*t).
      The inputs
% are dt, the time step size, and tfinal, the final simulation
      time.

N = ceil(tfinal/dt);     %Determine the number of time steps
y = zeros(N);            %Initialize y to hold the numerical
      solution
y_exact = zeros(N);      %Initialize y_exact to hold the exact
      solution
time = zeros(N);         %Initialize time to hold the times

y(1) = 1;                %Initial conditions
y_exact(1) = 1;
time(1) = 0;
```

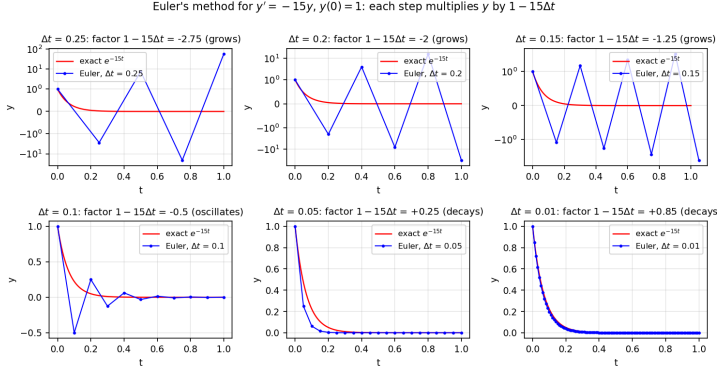
```
%Loop through all time steps and calculate y using Euler's
      method
for i = 2:N,
  y(i)= y(i-1)+dt*(-15*y(i-1));
  time(i)=(i-1)*dt;                  %update time
  y_exact(i) = exp(-15*time(i));     %save exact solution
end

%graph the numerical and the exact solutions
hold off
plot(time, y, '-b');
hold on
plot(time, y_exact, '-r');
xlabel('time');
ylabel('y');
legend('Eulers method', 'Exact solution');
```

Call it as `stiff_ode_example(dt, tfinal)` and try `(0.2, 1)`, `(0.1, 1)`, `(0.01, 1)` to observe the instability.

Lecture 19, slides 13–16. The repository's copy has two corrections (slide 11); `stiff_ode_example.py DT TFINAL` is the Python version.

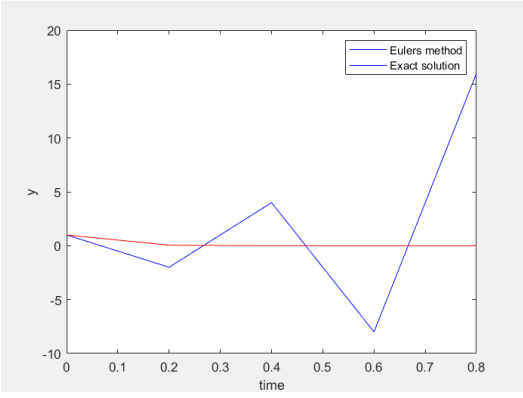
SIX TIME STEPS FOR $y' = -15y$



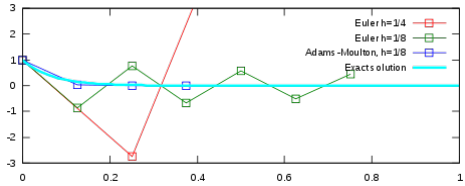
Δt	factor $1 - 15\Delta t$	behaviour	the lab's three	
0.25	-2.75	grows, alternating ($y_4 = 57$)		the hand calculation
0.2	-2	grows, alternating ($y = 16$ at $t = 0.8$, -32 at $t = 1$)	`(0.2, 1)`	"exits the range of the graph" (said of $h = 1/4$ on slide 6)
0.15	-1.25	grows slowly (± 4 at $t = 1$)		just outside $2/15$
0.1	-0.5	decays, changes sign every step	`(0.1, 1)`	"oscillates about zero" (said of $h = 1/8$)
0.05	$+0.25$	decays, too fast at first		
0.01	$+0.85$	follows e^{-15t}	`(0.01, 1)`	

`tools/ch4d_figures.py` with `stiff_ode_example.py`; the top row uses a symmetric log axis so that the growth fits.

THE COURSE'S OWN PICTURE, AND THE SKETCH IT STARTED FROM



Lecture 19, slide 17: MATLAB, `stiff_ode_example(0.2, 1)`. The blue curve reaches 16 at $t = 0.8$; the plot stops there because of the second bug of slide 11.



Lecture 19, slides 13 and 16: the sketch of the three behaviours (red grows, green oscillates, blue follows the cyan exact curve).

Read the MATLAB plot against the table: $\Delta t = 0.2$ gives the factor -2, so $y = 1, -2, 4, -8, 16$ at $t = 0, 0.2, \ldots, 0.8$. That is the whole phenomenon: not "errors accumulate", but "errors are multiplied by 2 every step".

TWO BUGS IN THE HANDED-OUT `stiff_ode_example.m`

- `zeros(N)` makes an $N \times N$ matrix, not a vector of length N. `y`, `y_exact` and `time` were all $N \times N$; the loop filled their first columns and `plot(time, y)` drew N curves, $N - 1$ of them the all-zero columns, i.e. N points on top of each other at the origin, invisible. The visible damage is to the legend, whose two labels go to the first two of the $2N$ line objects: in the course's plot (slide 10) "Exact solution" is drawn in the blue of the Euler curve and the red exact curve has no entry. The repository's copy uses `zeros(1, N)`.
- The time grid stops one step short of `tfinal`: with `N = ceil(tfinal/dt)` and the loop `for i = 2:N`, the last time is `(N-1)*dt`; for `(0.2, 1)` that is 0.8, as the picture shows. The copy uses `N = ceil(tfinal/dt) + 1` time levels so that the last is at (or just past) `tfinal`.

Neither changes the numbers that matter (the first column of `y` was right), which is how both survived unnoticed for years. The Python version (`stiff_ode_example.py`) is a vector version of the corrected file; Octave and Python agree to the last digit for $\Delta t = 0.05$ and 0.15 (the check is recorded in the README of `examples/support/matlab-exercises`, Appendix A; chapter 5 runs the same three cases).

> A good habit from this: check `size(y)` after the initialisation, and print the last time value before you trust a plot's right edge.

THE TRAPEZOIDAL METHOD DOES NOT HAVE THIS PROBLEM

The two-stage Adams–Moulton (trapezoidal) method of slide 6, applied to $y' = \lambda y$:

$$y_{n+1} = y_n + \tfrac{h}{2}\left(\lambda y_n + \lambda y_{n+1}\right) \quad \Longrightarrow \quad y_{n+1} = \frac{1 + h\lambda/2}{1 - h\lambda/2}\, y_n .$$

For $\lambda = -15$ the factor is $(1 - 7.5h)/(1 + 7.5h)$, which has modulus below 1 for *every* $h > 0$: the method is A-stable. With $h = 1/4$ the factor is -0.304 (it still changes sign, but decays); with $h = 1/8$ it is $+0.032$ (positive: no sign change any more, which is why the blue curve of the lecture's sketch, slide 10, hugs the exact one).

- The price: the new value appears on both sides (an implicit method). Here it is one division; for a nonlinear f it is a solve at every step.
- IB2d and IBAMR treat the fluid's viscous term implicitly (that is why the viscous time-step limit $\Delta t < \Delta x^2/(4\nu)$ does not bite) and the boundary force explicitly: the force is evaluated at the known positions and then the points are moved. The force is the stiff part, so it sets Δt (slide 13); an implicit IB method is a research topic, not a switch.
- Exercise 47: add the trapezoidal step to `stiff_ode_example` and repeat the three runs.

THE CFL CONDITION, AND WHY IT IS NOT WHAT LIMITS AN IB RUN

The Courant–Friedrichs–Lewy condition is a necessary condition for stability when certain PDEs (usually hyperbolic ones) are solved by finite differences. The general CFL condition for the n-dimensional case:

$$C = \Delta t \sum_{i=1}^{n} \frac{u_{x_i}}{\Delta x_i} \leq C_{\max}, \qquad \text{typically } C_{\max} = 1 :$$

in one step, information may not travel further than one cell. In immersed boundary simulations we rarely have to worry about this. The lecture's reason (Lecture 12, "IBAMR instabilities"):

- instabilities typically occur when the time step is too big;
- the time step necessary for stability is inversely proportional to the force applied to the fluid;
- the forcing term is typically what causes the instability (not the CFL condition);
- strategy: avoid large forces by minimising accelerations and very stiff springs.

The lab checks the second point on the rubber band (slide 33): $\Delta t_{\max}$ does fall as the stiffness rises. The CFL number of a typical IB run is small anyway: at 512^2 with $\Delta t = 10^{-5}$ and $u \sim 1$, $C = 0.005$.

Lecture 19, slide 18; Lecture 12, slides 12–13.

CONVERGENCE

A numerical method is convergent if the numerical solution approaches the exact solution as the step size $h \to 0$. In the immersed boundary method you typically need to worry about convergence in space: is the flow around the boundary well resolved? Are shed vortices well resolved?

- Convergence is checked by running the same problem at two or three resolutions and watching a number you care about settle (a force, a flow rate, a position). The lab does this for the band's shape (slides 39 and 42).
- Stability and convergence are linked (for linear problems, consistency + stability = convergence, the Lax equivalence theorem), which is why an unstable scheme cannot be "fixed" by refining: the heat equation at $r > 1/2$ gets worse with every extra step, not better.
- The order (slide 22) tells how much a refinement buys: halving h divides the error by 2^p. The IB method is second order away from the boundary and first order at it (slide 26), so an IB answer typically converges with p between 1 and 2.

Lecture 19, slide 19; Lecture 12, slide 14.

THE HEAT EQUATION: A MODEL WITH BOTH ISSUES AT ONCE

To look at an example that incorporates issues of instability and convergence in space and time, we simulate the heat equation. Let $u(x,t)$ give the temperature at time t and position x along a bar:

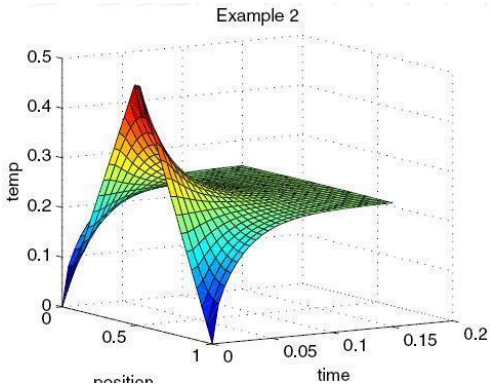
$$u_t = D\,u_{xx}, \quad 0 < x < 1,$$

$$u(0,t) = u(1,t) = 0, \quad u(x,0) = \sin 2\pi x,$$

with $D = 1$. Intuitively this is the heat distribution on a bar with hot spots; over time the heat diffuses. Exact solution: $u = e^{-4\pi^2 t} \sin 2\pi x$.

The scheme (explicit, forward in time, central in space): with $x_j = j\,\Delta x$, $\Delta x = 1/(n+1)$, $\Delta t = 0.1/T$,

$$u_j^{m+1} = r\,u_{j-1}^m + (1-2r)\,u_j^m + r\,u_{j+1}^m, \quad r = \frac{\Delta t}{\Delta x^2}.$$



Lecture 19, slide 21: $u(x,t)$ for a different initial condition (a single tent-shaped hot spot, followed to $t = 0.2$). The script plots the analytical and numerical solutions at a single time, $t = 0.1$, for n interior points and T time steps: `heat_equation(n, T)`.

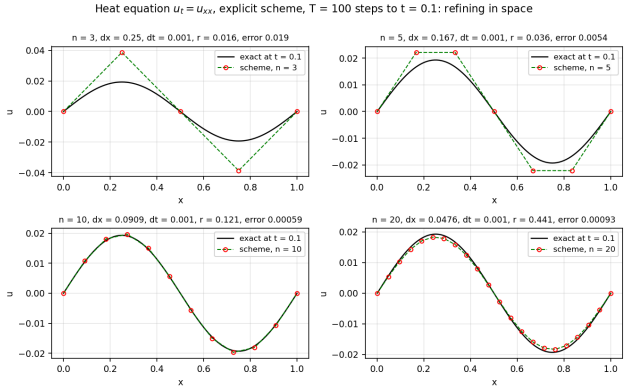
HEAT_EQUATION.M, AS HANDED OUT

```
function heat_equation(n,T)
% function Example4_1(n,T)
% solve Ut=Uxx, U(0,t)=U(1,t)=0, U(0,x)=sin(3*pi*x).
%    n: n+1 is the number of intervals in x-direction,
%    T: the no. of intervals in t-dir.
clf,
t=0.1; dt=t/T; dx=1/(n+1);
r=dt/(dx*dx);
x=[dx:dx:1-dx]; % all the inner points
%analytical solution when t=0.1:
ex=[0:0.005:1];
u_ex=exp(-4*(pi*pi)*t)*sin(2*pi*ex);
%up is the previous solution and un is the next solution.
up=zeros(size(x)); un=zeros(size(x));
%initial condition:
up=sin(2*pi*x);
```

```
%the calculating loop:
for m=1:T,
  for j=2:n-1,
    un(j) = r*up(j-1)+(1-2*r)*up(j)+r*up(j+1);
  end,
%take care of boundary conditions:
  un(1)=(1-2*r)*up(1)+r*up(2);
  un(n)=r*up(n-1)+(1-2*r)*up(n);
%put un into up:
  up=un;
end,
p=plot(ex,u_ex,[0 x 1],[0 up 0],'--',[0 x 1],[0 up 0],'o');
h=text(0.01,-0.017,sprintf('dx=%6.3g, dt=%g, r=%g',dx,dt,r));
set(p,'LineWidth',1.2), set(h,'FontSize',18),
print -deps Example4_1.eps
```

One thing fixed in the repository's copy: the header said $\sin 3\pi x$ (and `U(0,x)`) where the code and the exact solution use $\sin 2\pi x$ at $t = 0$. Two things left as they were, so you will meet them: the name in the comment, `Example4_1`, is from an older course, and `print -deps` writes `Example4_1.eps` into the current folder on every call (delete the line if you do not want the file). The Python version is `heat_equation.py N T` (vectorised loop, PNG output, the same numbers in the stable cases and to about 10^{-9} at $r = 0.57$; at $r = 0.8$ the blow-up grows from round-off, which differs from program to program: the README of `examples/support/matlab-exercises`, Appendix A; chapter 5 runs them).

CONVERGENCE IN SPACE: $T = 100$, $n = 3, 5, 10, 20$



n	Δx	r	max error
3	0.25	0.016	$1.9 \cdot 10^{-2}$
5	0.167	0.036	$5.4 \cdot 10^{-3}$
10	0.091	0.121	$5.9 \cdot 10^{-4}$
20	0.048	0.441	$9.3 \cdot 10^{-4}$

The lab: set $T = 100$ and try $n = 3, 5, 10, 20$ to show convergence in space. The curves do converge to the exact one; the error falls by 3.6 from $n = 3$ to 5 and by 9 from 5 to 10, more than the 2.3 and 3.4 a second-order scheme would give for those Δx ratios (slide 23 measures the clean $p = 2$ with r held fixed; here r is climbing towards the cancelling value $1/6$ of slide 24).

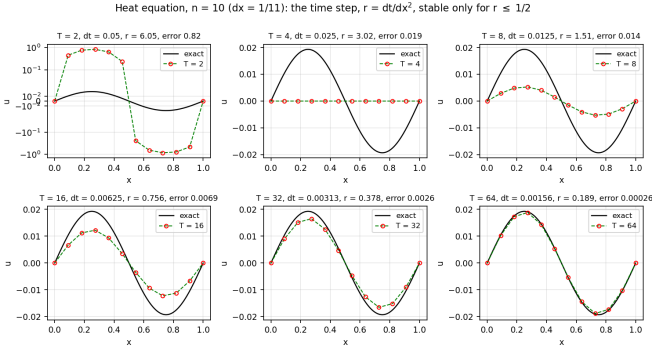
`tools/ch4d_figures.py` with `heat_equation.py`; the errors are the maximum over the interior points at $t = 0.1$.

THE SURPRISE: $n = 20$ IS WORSE THAN $n = 10$

The surprise: $n = 20$ is *worse* than $n = 10$. With Δt fixed at 10^{-3} the time error does not shrink ($\Delta t/2 = 5 \cdot 10^{-4}$), while the space error falls below it between $n = 10$ ($\Delta x^2/12 = 6.9 \cdot 10^{-4}$) and $n = 20$ ($1.9 \cdot 10^{-4}$); the two have opposite signs (slide 22: the leading error is $(\Delta t/2 - \Delta x^2/12)\, u_{xxxx}$), so they partly cancel at $n = 10$ and the time error is left standing at $n = 20$. Refining one of Δx, Δt alone does not always help; refine both.

> The table of slide 17: $5.9 \cdot 10^{-4}$ at $n = 10$, $9.3 \cdot 10^{-4}$ at $n = 20$, both with $\Delta t = 10^{-3}$. Slide 24 shows the same effect from the other side, with n fixed and Δt varied.

THE TIME STEP: $n = 10$, $T = 2, 4, 8, 16, 32, 64$



T	Δt	r	error	
2	0.05	6.05	0.82	a sine 44 times too large
4	0.025	3.02	0.019	wiped to $2 \cdot 10^{-6}$
8	0.0125	1.51	0.014	too small
16	0.00625	0.756	0.0069	unstable, not yet visible
32	0.0031	0.378	0.0026	stable
64	0.0016	0.189	0.00027	stable

The lab: set $n = 10$ and try $T = 2, 4, 8, 16, 32, 64$; the run goes to $t = 0.1$, so $\Delta t = 0.1/T$ and $r = 12.1/T$. Four of the six are above the limit $r = 1/2$. $T = 2$ keeps 85 % of the initial sine after two steps where 2 % should be left (its multiplier is -0.92 per step: slide 20), a smooth curve 44 times too large; $T = 4$ wipes the solution out (the sine's multiplier $1 - 4r\sin^2(\pi/11) = 0.04$ at $r = 3.02$: one step removes 96 % of it); $T = 8$ decays too fast (0.52 per step); $T = 16$ at $r = 0.756$ *looks* fine, and is not: slide 20.

`tools/ch4d_figures.py`; the first panel has a symmetric log axis.

WHY $r \le 1/2$, AND THE UNSTABLE RUN THAT LOOKS FINE

Put a single Fourier mode $u_j^m = g^m e^{i\theta j}$ into the scheme ($\theta = k\,\Delta x$):

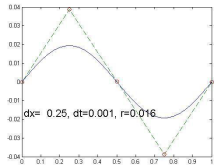
$$g = 1 - 2r + 2r\cos\theta = 1 - 4r\sin^2\tfrac{\theta}{2}, \qquad |g| \le 1 \text{ for all } \theta \iff r \le \tfrac{1}{2}.$$

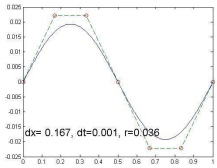
The most dangerous mode is the sawtooth $\theta = \pi$ (alternating signs from point to point), with $g = 1 - 4r$: it is multiplied by $|1 - 4r|$ every step.

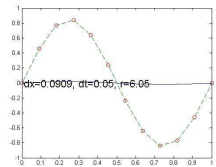
- $r = 6.05$ ($T = 2$): $g = -23$ for the sawtooth, and the initial sine, $\theta = 2\pi/11$, has $g = -0.92$: after two steps 0.85 of the sine is still there (the exact solution keeps 0.02), while the sawtooth, starting from round-off, has grown only $23^2 \approx 540$ times, to 10^{-13}. The first panel of slide 19 is therefore a clean sine 44 times too large, not noise; it would take a dozen steps at this r for the noise to reach order one ($23^{12} \cdot 10^{-16} \approx 2$).
- $r = 0.756$ ($T = 16$): $g = -2.02$ for the sawtooth. But the initial condition is a smooth sine, so the sawtooth starts at round-off, 10^{-16}; after 16 steps it has reached $2^{16} \cdot 10^{-16} \approx 10^{-11}$: invisible. Run the same r for 200 steps and it is 10^{45}. Unstable does not mean visibly wrong right away. The same happens in IB runs that "blow up at $t = 0.8$" after looking reasonable: the growth started at step 1.
- $r = 3.02$ ($T = 4$): $g = -11$ for the sawtooth and $g = 1 - 4r\sin^2(\pi/11) = 0.04$ for the sine: the sine itself is all but wiped out in one step ($0.04^4 = 2.5 \cdot 10^{-6}$ after four), which is why that panel is flat.

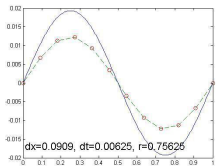
The rule for the explicit heat equation, $\Delta t \le \Delta x^2/(2D)$, is the one IB2d and IBAMR avoid by treating viscosity implicitly; the boundary force is the term they treat explicitly, and it has an analogous limit (slide 33).

THE COURSE'S MATLAB PLOTS

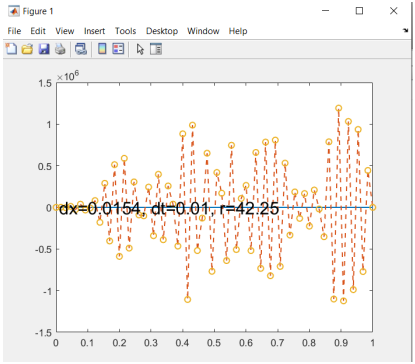








Lecture 19, slides 25–26: $n = 3$ and 5 at $T = 100$ (top), $T = 2$ and $T = 16$ at $n = 10$ (bottom); the text in each panel is the script's own `dx, dt, r` stamp.



Lecture 19, slide 27: a blow-up to $\pm 1.2 \cdot 10^6$ (the axis runs to $\pm 1.5 \cdot 10^6$) at $r = 42.25$ ($n = 64$, $T = 10$): the sawtooth mode, alternating from point to point.

The bottom-left panel ($T = 2$, $r = 6.05$) is the same as our first panel of slide 19 on a linear axis; the bottom-right ($T = 16$) is the "unstable but not yet visible" case.

RATE AND ORDER OF CONVERGENCE

In numerical analysis, the speed at which a convergent sequence approaches its limit is called the rate of convergence. Suppose that the sequence $\{x_k\}$ converges to the number L. It converges linearly to L if there is a number $\mu \in (0,1)$ with

$$\lim_{k\to\infty} \frac{|x_{k+1} - L|}{|x_k - L|} = \mu;$$

μ is called the rate of convergence. The sequence converges with order q to L, for $q > 1$, if

$$\lim_{k\to\infty} \frac{|x_{k+1} - L|}{|x_k - L|^q} = \mu, \qquad \mu > 0;$$

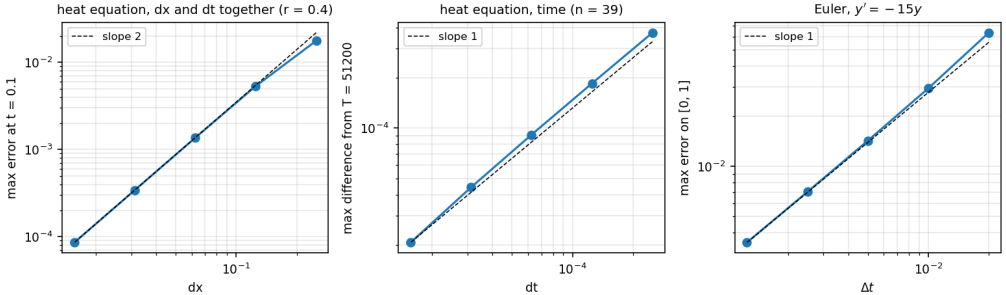
$q = 2$ is quadratic convergence, $q = 3$ cubic (Newton's method is quadratic; that is what these definitions are for).

For a discretisation the word is used differently: a method has order p when the error is $O(h^p)$. Measure it with three resolutions: $e_h \approx Ch^p$ gives

$$p \approx \log_2 \frac{e_h}{e_{h/2}}, \qquad \text{or, without an exact solution,} \qquad p \approx \log_2 \frac{|x_h - x_{h/2}|}{|x_{h/2} - x_{h/4}|}.$$

The explicit heat scheme has truncation error $\tau = \left(\frac{\Delta t}{2} - \frac{\Delta x^2}{12}\right) u_{xxxx} + \ldots$ (using $u_{tt} = u_{xxxx}$): first order in time, second in space, and the two cancel at $r = 1/6$.

MEASURING THE ORDER ON THE TWO WARM-UPS



heat, $r = 0.4$ fixed: Δx	1/4	1/8	1/16	1/32	1/64	p = 1.7, 2.0, 2.0, 2.0
max error	$1.8 \cdot 10^{-2}$	$5.3 \cdot 10^{-3}$	$1.4 \cdot 10^{-3}$	$3.4 \cdot 10^{-4}$	$8.6 \cdot 10^{-5}$	
heat, $n = 39$: Δt	$2.5 \cdot 10^{-4}$	$1.25 \cdot 10^{-4}$	$6.25 \cdot 10^{-5}$	$3.1 \cdot 10^{-5}$	$1.6 \cdot 10^{-5}$	p = 1.0, 1.0, 1.05, 1.1
difference from $T = 51200$	$3.7 \cdot 10^{-4}$	$1.9 \cdot 10^{-4}$	$9.1 \cdot 10^{-5}$	$4.4 \cdot 10^{-5}$	$2.1 \cdot 10^{-5}$	
Euler, $y' = -15y$: Δt	0.02	0.01	0.005	0.0025	0.00125	p = 1.1, 1.05, 1.0, 1.0
max error on $[0, 1]$	$6.4 \cdot 10^{-2}$	$2.9 \cdot 10^{-2}$	$1.4 \cdot 10^{-2}$	$7.0 \cdot 10^{-3}$	$3.5 \cdot 10^{-3}$	

Three ways to measure: against the exact solution (left, right), or against a much finer run when there is none (middle), which is how an IB run is measured.

THE $r = 1/6$ CANCELLATION, WITH NUMBERS

The same script, $n = 79$ ($\Delta x = 1/80$), three time steps:

T	Δt	r	max error at $t = 0.1$
2000	$5 \cdot 10^{-5}$	0.32	$3.6 \cdot 10^{-5}$
4000	$2.5 \cdot 10^{-5}$	0.16	$1.6 \cdot 10^{-6}$
8000	$1.25 \cdot 10^{-5}$	0.08	$2.0 \cdot 10^{-5}$

Halving Δt from 0.32 to 0.16 improves the answer twenty-fold; halving it again makes it worse. Nothing is wrong: at $r = 1/6$ the first-order time error and the second-order space error cancel, and the scheme is fourth order for that one ratio. The same effect produced the "$n = 20$ worse than $n = 10$" row of slide 17 ($r = 0.44$ against 0.12): with $n = 20$ and $T = 300$ ($r = 0.147$) the error is $6.8 \cdot 10^{-5}$, fourteen times better than at $T = 100$, and at $T = 1000$ ($r = 0.044$) it is $4.2 \cdot 10^{-4}$ again.

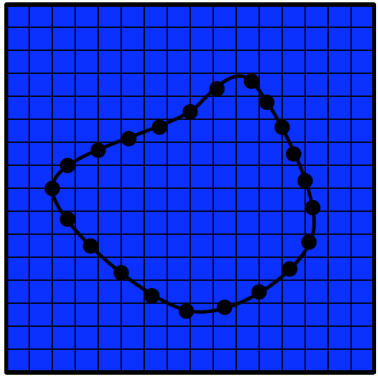
> Lesson for the IB runs: when a refinement makes a number worse, do not conclude that the code is wrong or that the coarse run was right. Error terms of opposite sign exist; look at three resolutions, not two, and refine space and time together.

STABILITY AND CONVERGENCE IN IB2D: DESCRIBING THE BOUNDARY

In the immersed boundary method, boundaries are always elastic:

- they can be stiff enough that they act like rigid objects;
- they can be moved in prescribed ways using target points (chapter 4c);
- boundaries have been modelled to resist bending, stretching and twisting;
- all of these properties can change in time in prescribed ways or according to other models.

Immersed boundary approach to fluid–structure interaction. Discretise the Eulerian equations on a Cartesian grid and solve for velocity and pressure, $u(x,t)$, $p(x,t)$, with the force density $f(x,t)$; discretise the Lagrangian equations on a curvilinear mesh and solve for force density and position, $F(r,t)$, $X(r,t)$. There is no body-conforming moving grid for the fluid: that is what makes the method simple, and what makes its resolution questions particular.



Lecture 19, slide 32: the Lagrangian points of a closed boundary on the Eulerian grid.

Lecture 19, slides 30–32 (the speaker notes: “now let’s move to what we can call numerical experiments”).

THE RULES OF THUMB

- Typically you want 2 boundary points for every fluid grid point: $\Delta s = \Delta x/2$. (Fewer, and fluid can pass between the points; IB2d's driver assumes this spacing when it spreads forces, slide 37.)
- Second order in space if the boundaries are smooth, first order if there are edges; and first order at the boundary itself, where the regularised delta function smooths everything over 3–4 cells.
- For publication-quality results, typically 256×256 or 512×512 grids for the fluid domain (2D).
- The time step is first or second order, depending on the version (the lab's time-step study could not measure it: slide 42).
- Typically $\Delta t = 10^{-5}$ or 10^{-6} when the characteristic time is $t = 1$ and the grid is 512×512: for example, one wing stroke every 1 unit of time.

What this chapter adds to the list, from the runs: (1) the time step a band tolerates falls with its stiffness, between $k^{-1/2}$ and $1/k$ here (a factor 1.3–4 for ten times the stiffness from the shipped band up, at least 8 from the ten-times-softer band to it); (2) in IB2d a spring constant is a force density and the driver weights it with $L_x/(2N_x)$, so refining the grid with the same k makes a *softer* band: scale k with N^2 to keep the physics (and with the number of points when you add points to a grid); (3) a closed band loses enclosed area at a rate set by the grid, not by the points per cell (13 % by $t = 1$ at 32^2, 4 % at 256^2): the standard IB method is not volume-conserving, and conserved quantities are the cheapest convergence check.

Lecture 19, slide 33; Lecture 12, slide 21.

THE LAB'S EXAMPLE: THE SHIPPED RUBBER BAND, AS IT IS

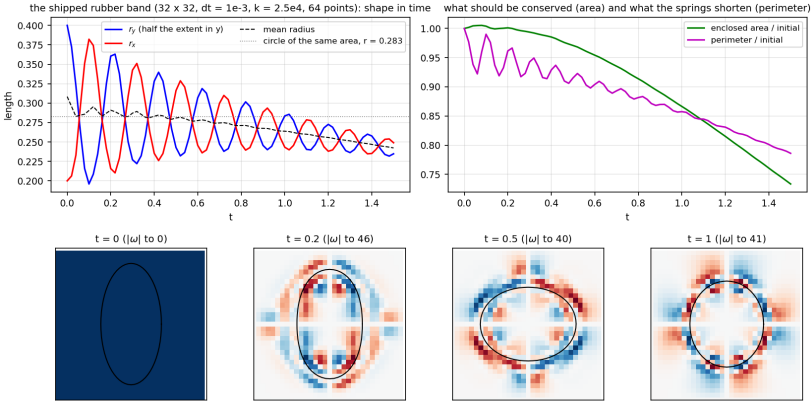
`pyIB2d/Examples/Rubberband_with_Springs` (chapter 4a), which is also `matIB2d/Examples/Example_Standard_Rubberband/Rubberband_with_Springs` (the course's `Rubberband_HW5` sat in the same folder, with its own generator: slide 31):

- a 1×1 periodic box, 32×32 cells, $\mu = 0.01$, $\rho = 1$, $\Delta t = 10^{-3}$, $T = 1.5$, a dump every 20 steps;
- 64 points on an ellipse centred at $(0.5, 0.5)$; the generator is called with $(a, b) = (0.4, 0.2)$ but its function takes `(N, rmin, rmax)`, so the 0.4 semi-axis is along y: the band is tall, not wide;
- springs between neighbours, $k = 2.5 \cdot 10^4$, resting length 0: the band wants to shrink; the fluid inside (incompressible) stops it, so it should settle to the circle of the same area, $r = \sqrt{0.08} = 0.283$;
- the perimeter is 1.94, so $\Delta s = 0.030 = 0.97\,\Delta x$: one point per cell, although the file says "2x resolution of the Eulerian grid" ($N = 2N_x$ would be two per cell for a perimeter of 1). The driver does not look: its spreading weight is `ds = Lx/(2.*Nx)` $= 0.0156$ (line 259 of `IBM_Driver.py`; MATLAB's line 188 takes the smaller of the two directions), two points per cell assumed.

```
cd pyIB2d/Examples/Stability_Lab
python3 Rubberband.py              # the shipped files, to 1e
      -16
python3 Rubberband.py 64 1e5       # 64 x 64 grid, k = 1e5
python3 Rubberband.py 32 1e5 0 4   # 32 x 32, two points per
      cell
python3 main2d.py                  # one run
python3 analyze.py                 # series.csv: shape in time
python3 sweep.py base blowup map   # the chapter's run groups
```

`examples/IB2d/Stability_Lab` in the repository: the generator with the grid, stiffness, resting length and points per cell as arguments, the sweep that made every table here (one folder per run under `runs/`), and the analysis (centroid, half-extents r_x, r_y, mean radius, perimeter, enclosed area from the `lagsPts` dumps). Dumps are written every 0.02 time units whatever Δt is (every step when $\Delta t > 0.02$), each one step late (slide 30), so runs compare dump by dump to within one step.

WHAT THE BAND DOES



The tall ellipse snaps to a wide one and back, seven to eight times in 1.5 units of time (a period of about 0.2: r_y has minima at $t = 0.10, 0.32, 0.52, \ldots, 1.48$), with decaying amplitude (r_y from 0.40 to 0.196 at $t = 0.10$). The mean radius falls below the area-preserving 0.283 because the enclosed area is not conserved: 97 % at $t = 0.5$, 87 % at $t = 1$, 73 % at $t = 1.5$. The fluid leaks through a band with one point per cell, and the springs (resting length 0) pull the perimeter in by 21 %. The four vortices at each “corner” are the flow the snapping makes. Run time: 3.7 s for 1500 steps (a desktop PC under WSL, Python).

BLOW-UP 1: A TIME STEP OF 1

The course's lab (MATLAB, `Rubberband_HW5`): open `input2d`, change `dt` to `1.0` and `Tfinal` to `100`, save and run `main2d`:

```
Temporal_Information {
Tfinal = 100.0              % Final time of
     simulation (s)
dt = 1.0             % Time-step (s)
}
```

Instability error!

```
Error using please_Move_Lagrangian_Point_Positions (line 84)
BLOW UP! (*forces TOO large*) -> try decreasing the time-step
or decreasing material property Stiffnesses
```

(In the 2024 code the message comes from lines 95 and 109 of that file: a `try`/`catch` around the same index look-up that the Python port does unguarded; a `NaN` or `Inf` position makes an invalid index, MATLAB throws, and the `catch` prints the message.) Then change the temporal information back: `Tfinal = 2.0`, `dt = 1.0e-3`.

BLOW-UP 1 IN PYTHON

Python (`sweep.py blowup`, case `dt1_T100`): the port has no such check. One thing to know first: the port writes dump k after step $k\cdot$ `print_dump` $+\,1$, one step late (its save test runs on the step counter before the counter is incremented, `IBM_Driver.py` line 834; harmless at $\Delta t = 10^{-3}$, visible here at $\Delta t = 1$). The points are spread over the whole box (each moved by $\Delta t\,u$, then wrapped by the periodic box): dump 1, written after step 2, has $r_x = r_y = 0.50$ and a perimeter of 39; dump 2, after step 3, holds only zeros (positions so large that $x \bmod L_x$ is exactly 0 in floating point); dump 3, after step 4, holds only `NaN` (coincident points give $0/0$ in the spring direction, `please_Find_Lagrangian_Forces_On_Eulerian_grid.py` line 539); in step 5 a `NaN` position is cast to an index:

```
File ".../IBM_Blackbox/Supp.py", line 101, in
      please_Move_Lagrangian_Point_Positions
    distX = give_Eulerian_Lagrangian_Distance(x[xInds],
      xL_H_ReSize, Lx)
IndexError: index -9223372036854775808 is out of bounds for
      axis 0 with size 32
```

That number is what `int(NaN)` gives. Whenever a Python IB2d run ends with this `IndexError`, read it as MATLAB's "BLOW UP": the time step is too large for the forces (or the forces are too large for the time step), and the first thing to try is the same: halve Δt or soften the stiffness.

`runs/dt1_T100/run.log`; `IBM_Driver.py` line 834.

BLOW-UP 2: A BAND 1000 TIMES STIFFER

The course's lab, with Δt back at 10^{-3}: "there is an inverse relationship between the force the boundary applies to the fluid and the time step required." Open `generate_mesh2d.m`:

```
mesh_name = 'rubber_band';          % structure name
kappa_spring = 2.0e3;               % spring constant (Newton)
```

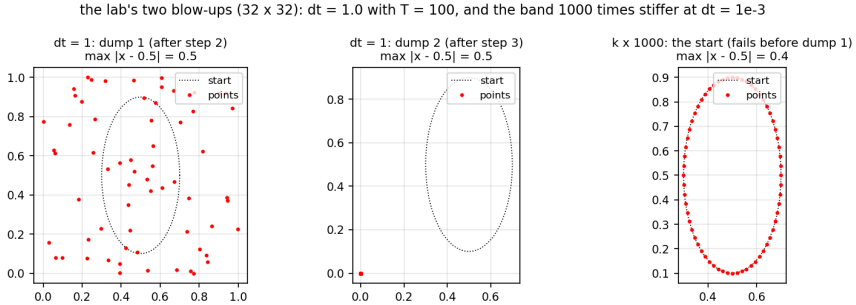
Change `kappa_spring` to `2.0e6`, save, run `generate_mesh2d`, run `main2d`:

```
Error in IBM_Driver (line 794)
    [xLag_h, yLag_h] = please_Move_Lagrangian_Point_Positions(mu,
       U, V, xLag, yLag,
    xLag, yLag, x, y, dt/2, grid_Info,0,poroelastic_Yes,
      poroelastic_info,F_Poro);
```

(the same BLOW UP, reported one call higher). Change `kappa_spring` back to `2.0e3`, save, run `generate_mesh2d` again.

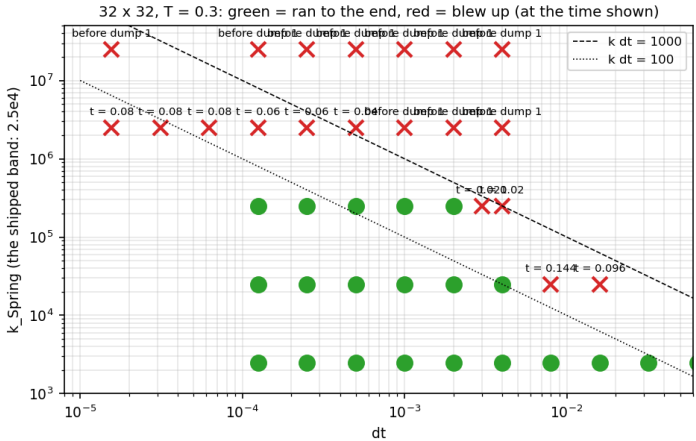
The course's `generate_mesh2d.m` is the IBAMR-style generator of chapter 6 adapted to IB2d (the lines the lecture shows are its, comment for comment; how it converts κ into IB2d's per-link constant is not on the slides), and its band is that generator's, not the shipped ellipse: a circle of radius 0.1 (with, if the geometry lines were kept, springs at their resting length and a small stretch in x), which is what the VisIt picture of slide 44 shows. The shipped IB2d band uses `Rubberband.m` with `k_Spring = 2.5e4` directly, and that is the band of this chapter's lab.

BLOW-UP 2 IN PYTHON, AND WHAT THE TWO BLOW-UPS LOOK LIKE



Python: `python3 Rubberband.py 32 2.5e7` then `main2d.py` (case `k1000x`): the same `IndexError` before the first dump (step 21 at this Δt), and the run takes no longer than the port's start-up (1.3 s, against 1.5 s plus 1.2 ms per step for the runs that live), so within the first few steps; not one dump beyond the initial one. Both blow-ups take a second of computing.

THE INVERSE RELATIONSHIP, MEASURED: 41 RUNS



Stiffness $\times$ time step on the 32^2 grid, each run to $T = 0.3$ (`sweep.py map map2`). Green: ran to the end. Red: the `IndexError`, at the time of the last dump written ("step 1" in the figure means that no dump beyond the first was written, i.e. failure before $t = 0.02$; slide 34 says how early). The dotted and dashed lines are $k\,\Delta t = 100$ and 1000.

THE MAP AS NUMBERS

k	relative to the shipped band	survives up to Δt	fails at	$k\,\Delta t$ at the edge
$2.5 \cdot 10^3$	$\times 0.1$	0.064 (every value tried)	–	> 160
$2.5 \cdot 10^4$	$\times 1$	0.004	0.008 ($t = 0.14$), 0.016 ($t = 0.10$)	100–200
$2.5 \cdot 10^5$	$\times 10$	0.002	0.003 and 0.004 ($t = 0.02$)	500–750
$2.5 \cdot 10^6$	$\times 100$	none	before the first dump ($t < 0.02$) at $\Delta t = 0.004$, 0.002, 0.001; $t = 0.04$ at $5 \cdot 10^{-4}$, 0.06 at 2.5 and $1.25 \cdot 10^{-4}$, 0.08 at $6.25 \cdot 10^{-5}$ down to $1.6 \cdot 10^{-5}$	–
$2.5 \cdot 10^7$	$\times 1000$	none	before the first dump ($t < 0.02$) for every Δt: within the first few steps for $\Delta t \geq 1.25 \cdot 10^{-4}$ (wall time = start-up), after some 1100 steps at $1.6 \cdot 10^{-5}$ (1.4 s of stepping)	–

Read along a row: the shipped band takes a step of 0.004 and not 0.008; ten times stiffer, 0.002 and not 0.003. Read down a column: at $\Delta t = 10^{-3}$ the band may be ten times stiffer than shipped, not a hundred. The run time to $T = 0.3$ at 32^2 is 1.4–4.5 s whatever happens (6 and 10 s for the two smallest steps, $3 \cdot 10^{-5}$ and $1.6 \cdot 10^{-5}$), so a map like this costs minutes, and it is the cheapest way to know where an example of your own stands.

READING THE MAP

- The lecture's statement holds in direction: the stiffer the band, the smaller the admissible step. The exponent is not clean. From the shipped band to the one ten times stiffer, $\Delta t_{\max}$ falls by a factor between 1.3 and 4 (the edges lie in $[0.004, 0.008)$ and $[0.002, 0.003)$), closer to $k^{-1/2}$, the scaling of a mass–spring system ($\Delta t < 2/\omega$, $\omega = \sqrt{k/m}$) with the fluid as the mass, than to $1/k$; from the band ten times softer to the shipped one it falls by at least 8–16 (0.064, the largest step tried, still ran at $k \times 0.1$), which is $1/k$ or steeper. At the edge, $k\,\Delta t$ is > 160, 100–200 and 500–750: of order 10^2–10^3, not a constant.
- The band 100 times stiffer is a different story: for every Δt from $5 \cdot 10^{-4}$ down to $1.6 \cdot 10^{-5}$ it fails at nearly the same moment (between $t = 0.04$ and 0.10, later for the smaller steps), and for $\Delta t \geq 10^{-3}$ before the first dump. Its dumps up to then are not the fast snap a band a hundred times stiffer should show (ten times the speed of the shipped band, by the $T^{-1/2}$ scaling of slide 37): r_y falls only from 0.40 to 0.29 by $t = 0.08$, slower than the shipped band's 0.22, while the perimeter is already 11 % and the enclosed area 7 % down. A smaller step does not save it, so the step is not what limits it; whether a finer grid does, which would say the failure is the grid's (the fluid at those speeds on 32 cells), is exercise 6 of slide 47.
- The band 1000 times stiffer fails before its first dump at every Δt tried, down to $1.6 \cdot 10^{-5}$: within the first few steps for $\Delta t \geq 1.25 \cdot 10^{-4}$ (the run takes no longer than the start-up), after some 1100 steps at $1.6 \cdot 10^{-5}$ (1.4 s of stepping at 1.2 ms per step). With resting length 0 the tension in each of its links is $k\,\Delta s \approx 7.5 \cdot 10^5$ (the net force on a point is the difference of its two links' pulls), and a step short enough to live with it was not found.
- Lecture 12's strategy stands: avoid large forces (large accelerations, very stiff springs); a stiff band costs time steps as well as steps.

REFINING THE GRID, THE COURSE'S WAY

`generate_mesh2d.m`:

```
L = 1;                              % length of computational
      domain (m)
N = 32;                             % number of Cartesian grid
      meshwidths at the finest level of the AMR grid
dx = L/N;                           % Cartesian mesh width (m)
```

"This is a pretty coarse simulation since there are only 32 spatial grid points in each direction. Let's change it to 64." Change `N` to `64`.

```
npts = ceil(2*(band_length/L)*N);   % number of points along the
      rubber band
ds = band_length/(npts);            % physical distance between
      neighboring Lagrangian mesh points (m)
```

"This will set `ds` to approximately `dx/2`." Run the code to generate new files.

`input2d`: "you also need to update the grid size."

```
Grid_Parameters {
Nx = 32                  % # of Eulerian Grid Pts
     . in x-Direction (MUST BE EVEN!!!)
Ny = 32                  % # of Eulerian Grid Pts
     . in y-Direction (MUST BE EVEN!!!)
```

Change `Nx` and `Ny` to `64`, save and run `main2d`.

> Two files must agree: the generator's grid (it sets Δs from Δx) and `input2d`'s. The shipped generators say so in capitals ("MAKE SURE MATCHES IN input2d"), and nothing checks it: a band generated for 64 and run on 32 has four points per cell and a different stiffness per unit length. The lab's `sweep.py` writes both from one number.

WHAT REFINEMENT DOES TO THE BAND IN IB2D

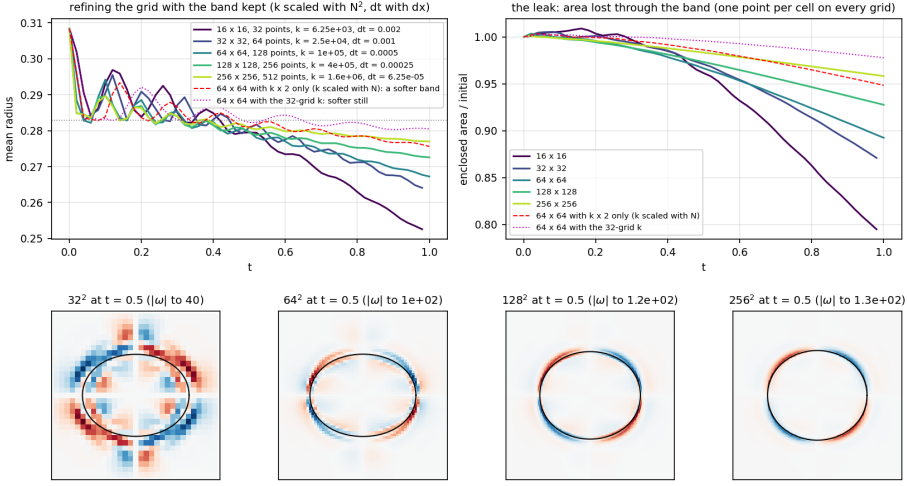
IB2d spreads each point's force to the grid as a line integral, $f(x) = \sum_i F_i \, \delta_h(x - X_i) \, \Delta s$ (`Fx = ((fx*ds) * delta_Y) @ delta_X`), with the driver's own $\Delta s = L_x/(2N_x) = \Delta x/2$, not the spacing of your file. Each spring's force is $k(|d| - L)$ with $|d| \approx \Delta s_{\text{file}}$ when $L = 0$, so one link gives the fluid $k \, \Delta s_{\text{file}} \, \Delta x/2$, and the band's effective tension is $T \propto k \, \Delta s_{\text{file}} \, \Delta x$. Two consequences:

- refining the grid ($\Delta s_{\text{file}} \propto \Delta x$, as the generators do) with the same k makes the band four times softer per doubling: keep $k \, \Delta x^2$, i.e. scale k with N^2;
- adding points on the same grid with the same k halves the tension: scale k with the number of points.

what was changed	k	T (shipped band = 1)	first r_y minimum at	period	
64^2, same file k (the naive refinement)	$2.5 \cdot 10^4$	$1/4$	$t = 0.20$	$\times 2$	softer still
64^2, $k \propto N$ ("keep $k \, \Delta s$")	$5 \cdot 10^4$	$1/2$	$t = 0.14$	$\times 1.4$	softer
64^2, $k \propto N^2$	10^5	1	$t = 0.10$	$\times 1$	the same band
32^2, twice the points, $k \times 4$	10^5	2	$t = 0.06$–0.08	$\times 0.7$	stiffer
32^2, twice the points, $k \times 2$	$5 \cdot 10^4$	1	$t = 0.10$	$\times 1$	the same band

The oscillation period is the check: it scales as $T^{-1/2}$, like a mass on a spring; only the rows with $T = 1$ keep the first minimum at $t = 0.10$, on every grid from 32^2 to 256^2 (next slide). Which row the course's recipe is depends on a line the lecture does not show: if its `generate_mesh2d.m` kept the IBAMR original's $\kappa \, \Delta s/\Delta s^2$, the file constant doubles when Δs halves and the recipe is the $k \propto N$ row. The IBAMR convention differs (chapter 6: the file's spring constant is $K/\Delta s$, a tension per unit strain divided by the spacing, which the generators write as $\kappa \, \Delta s/\Delta s^2$); whichever code, decide what must stay fixed under refinement and write the generator so that it does.

REFINING WITH THE BAND KEPT: 16^2 TO 256^2



`sweep.py grid2` (solid: $k = 2.5 \cdot 10^4\ (N/32)^2$, $\Delta t = 10^{-3} \cdot 32/N$, halved again at 256^2); the dashed and dotted 64^2 runs are the two wrong refinements of the previous slide.

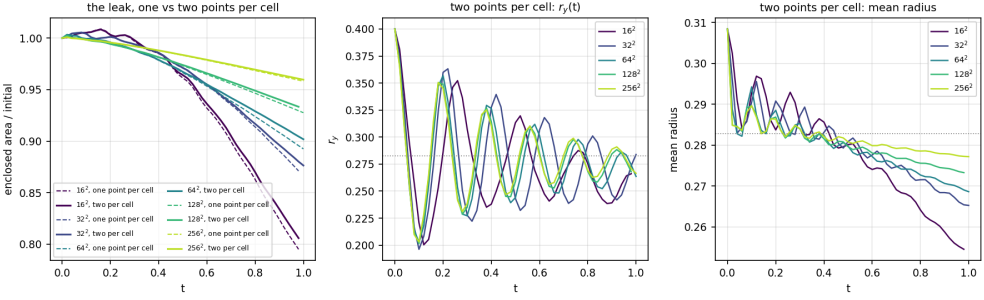
THE NUMBERS AT $t = 0.98$, AND THE LEAK

grid	points	k	Δt	first r_y min	area left	wall
16^2	32	$6.25 \cdot 10^3$	$2 \cdot 10^{-3}$	0.200 at 0.12	79.5 %	1.9 s
32^2	64	$2.5 \cdot 10^4$	10^{-3}	0.196 at 0.10	87.1 %	2.9 s
64^2	128	10^5	$5 \cdot 10^{-4}$	0.203 at 0.10	89.6 %	8.6 s
128^2	256	$4 \cdot 10^5$	$2.5 \cdot 10^{-4}$	0.201 at 0.10	93.0 %	38 s
256^2	512	$1.6 \cdot 10^6$	$6.25 \cdot 10^{-5}$	0.203 at 0.10	95.9 %	384 s

mean radius at $t = 0.98$: 0.2526, 0.2641, 0.2677, 0.2727, 0.2771, climbing towards the area-preserving 0.283 as the leak closes; successive differences 0.0115, 0.0036, 0.0050, 0.0043.

- The band's dynamics are converged from 32^2 on: the first minimum (depth 0.196–0.203, time 0.10) agrees to 2 % on every grid from 32^2 on (16^2 is late, at $t = 0.12$).
- Its area is not: 4–20 % of the enclosed fluid is gone by $t = 1$, and the loss falls only slowly with resolution (by a fifth to two fifths per doubling, 20.5, 12.9, 10.4, 7.0, 4.1 %: an order between 0.3 and 0.8). That is why r_y and the mean radius at a fixed late time do not line up as a clean p: the leak, not the discretisation of the dynamics, dominates the late-time numbers.
- Two points per cell do not change it (next slide): it is the method's volume error, which only the grid reduces.

TWO POINTS PER CELL: THE SAME BAND, THE SAME AREA LOSS



grid	points	k	area left at $t = 0.98$	same, one point per cell	first r_y min
16^2	64	$1.25 \cdot 10^4$	80.6 %	79.5 %	0.200 at 0.12
32^2	128	$5 \cdot 10^4$	88.0 %	87.1 %	0.196 at 0.10
64^2	256	$2 \cdot 10^5$	90.5 %	89.6 %	0.203 at 0.10
128^2	512	$8 \cdot 10^5$	93.3 %	93.0 %	0.202 at 0.10
256^2	1024	$3.2 \cdot 10^6$	96.1 %	95.9 %	0.203 at 0.10

`sweep.py grid4`: twice the points on every grid, k doubled with them (slide 37), Δt halved (except at 256^2, where both series use $6.25 \cdot 10^{-5}$); dashed: the one-point-per-cell series of slide 39.

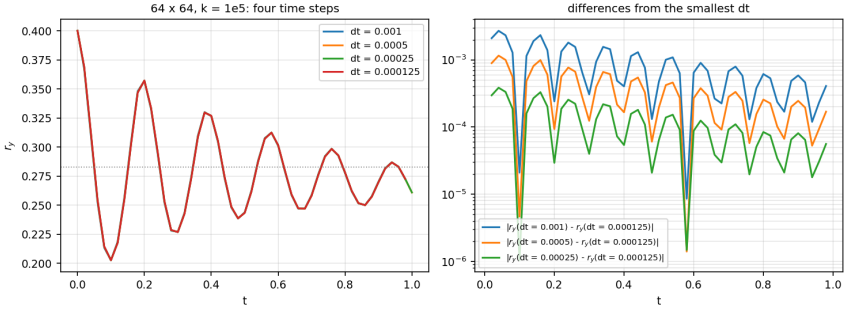
WHAT THE TWO-POINT SERIES SAYS

`sweep.py grid4`: twice the points on every grid, k doubled with them (slide 37), Δt halved (except at 256^2, where both series use $6.25 \cdot 10^{-5}$). It is the same band (first minimum 0.196–0.203 at $t = 0.10$ from 32^2 on), and it loses the same area as the one-point-per-cell band, to within one percent on every grid.

So the loss is not fluid passing between points. It is the standard IB method's volume error (Peskin and Printz 1993): the velocity interpolated to the points is not exactly divergence-free along the boundary, so a closed boundary gains or loses area at a rate set by the grid: 20, 13, 10, 7, 4 % at $t = 1$ from 16^2 to 256^2, an order between 0.3 and 0.8, unevenly.

Remedies: refine; a volume-conserving variant (Peskin–Printz's modified divergence; IBAMR's staggered-grid scheme does better: B. E. Griffith, "On the volume conservation of the immersed boundary method", Commun. Comput. Phys. 12 (2012) 401–432; chapter 5 describes the scheme); or measure what does not depend on the enclosed area. The two-points-per-cell rule still holds for what it is for: a boundary that must not let fluid *through* it (a wall with target points, chapter 4c).

THE TIME STEP, AT 64^2



Δt	10^{-3}	$5 \cdot 10^{-4}$	$2.5 \cdot 10^{-4}$	$1.25 \cdot 10^{-4}$	differences of $r_y(0.98)$	p
r_y at $t = 0.98$	0.272311	0.272550	0.272664	0.272719	$2.4 \cdot 10^{-4}$, $1.1 \cdot 10^{-4}$, $5.6 \cdot 10^{-5}$	1.07, 1.02
wall	5.8 s	8.5 s	14.3 s	23.4 s		

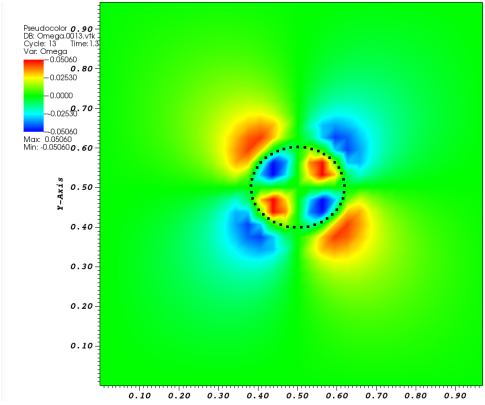
`sweep.py dt2`: 64^2, $k = 10^5$, $T = 1$. Halving Δt halves the difference, which looks like first order; but it is the dumps' lag, not the time stepping. The port writes every dump one step late (slide 30), so dump 49 of the $\Delta t = 10^{-3}$ run is the state at $t = 0.981$ and of the $1.25 \cdot 10^{-4}$ run at 0.98013; with r_y falling at 0.52 per unit time there, the lag alone accounts for $4.5 \cdot 10^{-4}$, $1.9 \cdot 10^{-4}$ and $6.5 \cdot 10^{-5}$ of the $4.1 \cdot 10^{-4}$, $1.7 \cdot 10^{-4}$ and $5.6 \cdot 10^{-5}$ measured against the finest run, and the differences change sign with the slope of r_y ($+5.9 \cdot 10^{-4}$ at $t = 0.90$, where r_y rises). What the four runs do show is that the time-stepping error is below the lag, under 10^{-4} in r_y at $\Delta t = 10^{-3}$; its order is not measured here (exercise 7). The four curves are indistinguishable on the left; the largest step is 10^{-3} of r_y away from the smallest.

WHAT REFINEMENT COSTS

grid	32^2	64^2	128^2	256^2	
cells	1024	4096	16384	65536	$\times 4$
points	64	128	256	512	$\times 2$
steps to $t = 1$ ($\Delta t \propto \Delta x$)	1000	2000	4000	16000	$\times 2$ (and $\times 4$ at 256^2)
wall (Python, desktop PC)	2.9 s	8.6 s	38 s	384 s	$\times 3$–10
per step	2.9 ms	4.3 ms	9.5 ms	24 ms	

- Each doubling of the grid costs $4\times$ the cells, $2\times$ the points and $2\times$ the steps: $16\times$ in the limit, $8\times$ in 2D when the fluid solve dominates. The 512^2 of the rules of thumb at $\Delta t = 10^{-5}$ is 10^5 steps per unit time: hours in MATLAB, a cluster job in Python as shipped.
- The Python port spreads forces by a dense delta-matrix product: two matrices of $N_b \times N_x$ and $N_y \times N_b$ (512×256 at 256^2), multiplied through every cell of the grid, $O(N_x N_y N_b)$ per step for a kernel that touches 16 cells per point; the fork branch `python-updates-2026` (pull request 34 to IB2d, October 2026) replaces it with the local four-point stencil, $O(16 N_b)$: the same numbers (the shipped band's dumps agree to 10^{-10}), $1.4\times$ faster on the 32^2 band, and a spreading cost that no longer grows with the number of cells.
- The time step shrank with Δx here because the stiffness grew with $1/\Delta s^2$ (slide 37) and, if $\Delta t_{\max}$ falls as $k^{-1/2}$ (slide 35), refining the grid with the same band halves the admissible step as well; the series ran at $\Delta t \propto \Delta x$ throughout, and exercise 5 measures whether that is the limit. That is the second half of "$\Delta t = 10^{-5}$ at 512^2".

WHAT THE COURSE SAW IN VISIT



Lecture 19, slide 37 (before the refinement slides 38–39): VisIt, Pseudocolor of `Omega` from `Omega.0013.vtk`, cycle 13, the MATLAB writer's stamp `Time: 1.3` (so the course's `input2d` dumped every 0.1 time units), automatic colour limits ± 0.0506, the band as about 40 dots on a circle of radius 0.1.

The homework's picture: the course's band with its four vortex pairs, the same quadrupole pattern as the panels of slide 28, but a different band. The dots lie on a circle of radius 0.1 about $(0.5, 0.5)$, about 40 of them (the IBAMR-style generator gives 41 points at $N = 32$ and 81 at 64, so this is the 32^2 run), and the vorticity is 0.05 against our 40: that is the IBAMR-style band of chapter 6 with $\kappa = 2 \cdot 10^3$ (a circle of radius 0.1 which, if the course's copy kept the original's geometry lines, starts from a small stretch with its springs at their resting length and relaxes), not the shipped ellipse with resting length 0 that snaps from tall to wide.

COMPARING TWO RESOLUTIONS IN VISIT

What to compare between two resolutions in VisIt (chapter 4b): the same cycle is *not* the same time when Δt changed (use the dump interval in time units: here a dump every 0.02); the colour limits must be set by hand to the same values; the band's `lagsPts` overlaid on `Omega` shows where the vorticity sits relative to the points, which is where a leak would show as flow through the band.

A PROTOCOL FOR AN EXAMPLE OF YOUR OWN

1. Decide what must stay fixed under refinement (tension, bending stiffness, target stiffness per unit length) and write the generator so that k follows Δs accordingly (slide 37); make the generator and `input2d` take the grid from one number.
2. Points at $\Delta x/2$ along every boundary that must not leak; check $\Delta s/\Delta x$ printed by the generator, not assumed from a comment.
3. Find the time step: start from a working example's Δt; if the run ends in `BLOW UP` or the `IndexError`, halve Δt (and, if it still fails at a tenth, suspect the forces or the grid, not the step: slide 35); then halve it once more and check that a number you care about moves by less than you care about (slide 42).
4. Refine the grid once with the band kept (k rescaled, Δt rescaled with Δx) and compare the same number at the same times: the dumps must land on the same times (set `print_dump` from Δt).
5. Report three resolutions, not two, before quoting an order (slide 24); report the one that converged and what it cost.
6. Conserved quantities are the cheapest check: the area inside a closed band, the mass of a tracer, the total circulation; a leak shows there long before it shows in a picture.

EXERCISES

1. The course's homework 5, part 1: run `stiff_ode_example` with $(0.2, 1)$, $(0.1, 1)$, $(0.01, 1)$ and explain each plot with the factor $1 - 15\Delta t$; then add the trapezoidal step (slide 12) and repeat.
2. Part 2: `heat_equation` with $T = 100$ and $n = 3, 5, 10, 20$, then $n = 10$ and $T = 2, \ldots, 64$; tabulate r and the maximum error; find the T at which $n = 20$ beats $n = 10$, and explain it with slide 24.
3. Part 3: the rubber band at $\Delta t = 1$ and at 1000 times the stiffness, in MATLAB or Python; quote the exact error line of your port.
4. Part 4: refine $32 \to 64$ the course's way (same κ in `generate_mesh2d.m`, both files updated) and the chapter's way (`sweep.py grid2`); compare the time of the first r_y minimum between the two and say which run is "the same band".
5. Extend the map (slide 33) along a line $k = 2.5 \cdot 10^4$ at 64^2 and 128^2: does $\Delta t_{\max}$ halve with Δx?
6. The 100-times-stiffer band fails by $t = 0.1$ at every Δt on 32^2 (before the first dump for $\Delta t \geq 10^{-3}$): run it at 64^2 and 128^2 with $\Delta t = 10^{-4}$ and report whether a finer grid takes it.

EXERCISES (2)

7. Measure the order in time of IB2d on the same band (64^2, four time steps), in Python or MATLAB (`analyze.py` reads the MATLAB dumps too), with the dumps' one-step lag removed (slide 42): compare states at the same step-corrected time, or compare the time of the first minimum of r_y interpolated from the dumps, or use the fork branch's final dump, which is written at exactly T_{final}: first or second?
8. Run the grid series one grid further (512^2, a cluster job: chapter 1) and add the row to the tables of slides 39 and 40.
9. The volume error: plot the area inside the band against time for 32^2 to 256^2 and fit the loss at $t = 1$ to a power of Δx; then run the same band in IBAMR (chapter 5's rubber band, `examples/IB/01-rubber-band`) at the same resolutions and compare.

PREPARING THE PROJECT GEOMETRY: THE COURSE'S HOMEWORK 6

The course's homework 6 ("you will share your geometry that you will use for your project"):

- 3D print or ANSYS: find a 3D object online (Thingiverse, GrabCAD) and explain what it is, or leave a note of anything you cannot find; create your own (SolidWorks, Autodesk Fusion); or mix: find an object and distort it. Share those objects.
- IB2d: geometries are created with a generator (`generate_mesh`, chapter 4c); write code that gives the x, y coordinates of the shape you want and submit it; if the object moves, give pseudo-code for how (the `update_*` functions of chapter 4c).

Appendix C collects the homework bank.

Lecture 19, slides 2 and 40.

SUMMARY

- Stability is a multiplier: Euler on $y' = -15y$ multiplies by $1 - 15\Delta t$ every step, the explicit heat scheme multiplies the sawtooth mode by $1 - 4r$; outside $|\cdot| \leq 1$ the run is garbage after a few steps, though it may look fine for a while if the bad mode starts at round-off.
- Convergence is measured, with three resolutions and a number you care about; the order says what a refinement buys, and error terms of opposite sign exist ($r = 1/6$).
- IB2d blows up the same way whatever the port: MATLAB says `BLOW UP`, Python says `IndexError: index -9223372036854775808`. Halve Δt or soften the force; the admissible step falls with the stiffness (between $k^{-1/2}$ and $1/k$ here), and a band too stiff or too fast for its grid fails at every Δt.
- A spring constant in IB2d is a force density: keep $k\,\Delta s\,\Delta x$ fixed when you refine ($k\,\Delta x^2$ when the points follow the grid, $k \times 2$ when you double the points on one grid), or you refine a different band. The oscillation period is the check.
- A closed IB band loses area: 13 % by $t = 1$ at 32^2, still 4 % at 256^2, and two points per cell do not change it; the standard method is not volume-conserving. Watch a conserved quantity, refine, or use a conserving variant.
- The Python IB2d's order in time is not settled by the lab: its dumps are one step late, and that lag, first order by construction, is all the time-step study resolves; refinement costs 8–$16\times$ per doubling and should halve the admissible step as well (exercise 5).
- Chapters 5 and 16 take the same questions to IBAMR; chapter 4e (the model gallery and muscles) is next in Part II.

REFERENCES AND CREDITS

- Sources merged (slide 3): “Lecture 19: Stability and convergence” (2021, edited 2023) and Lecture 12’s numerics block (2021–23), both from the UA course; the handed-out `stiff_ode_example.m` and `heat_equation.m` (the latter from an older numerical-methods course, “Example4_1”); the MATLAB pictures and VisIt screenshot are the lecture’s. Quotations are from those slides.
- The lab: `examples/IB2d/Stability_Lab` (`Rubberband.py`, `sweep.py`, `analyze.py`, `input2d`, `main2d.py`, README with every table) and `examples/support/matlab-exercises`; figures by `tools/ch4d_figures.py` and `ch4d_figures2.py`; the runs of 16 October 2026 (`wsl/final/ch4d_*.sh`) with the Python IB2d (clone of 2024-03-06, with the tracer fix of chapter 4b) on a desktop PC under WSL.
- IB2d: N. A. Battista, W. C. Strickland, L. A. Miller, “IB2d: a Python and MATLAB implementation of the immersed boundary method”, Bioinspir. Biomim. 12 (2017) 036003, and, with A. Barrett, “IB2d Reloaded: a more powerful Python and MATLAB implementation of the immersed boundary method”, Math. Meth. Appl. Sci. 41 (2018) 8455–8480; the spreading convention is in the code (`please_Find_Lagrangian_Forces_On_Eulerian_grid`).
- The immersed boundary method’s orders: C. S. Peskin, “The immersed boundary method”, Acta Numerica 11 (2002) 479–517; B. E. Griffith and C. S. Peskin, “On the order of accuracy of the immersed boundary method: higher-order convergence rates for sufficiently smooth problems”, J. Comput. Phys. 208 (2005) 75–105 (second order for sufficiently smooth problems, first order where the solution has a jump at the interface, as for a thin elastic boundary). The volume error: C. S. Peskin and B. F. Printz, “Improved volume conservation in the computation of flows with immersed elastic boundaries”, J. Comput. Phys. 105 (1993) 33–46.
- The warm-ups: any numerical-analysis text; e.g. R. J. LeVeque, *Finite Difference Methods for Ordinary and Partial Differential Equations* (SIAM, 2007), chapters 7 (absolute stability) and 9 (the heat equation, von Neumann analysis); the Wikipedia articles on numerical stability, the CFL condition and rate of convergence, which Lecture 19 quoted.